

Software Management: Experiment #2

Professor Bowen Du

2252709

Xuanhe Yang

May, 2025

Problem 1

Software Size Measurement

Experimental Background

Software size measurement is the foundation for software engineering management, software cost estimation, and software economic evaluation. Common software size measurement methods include Lines of Code (LOC), Function Point Analysis (FPA), Object Point, and Use Case Point methods. Among these, Function Point Analysis stands out as a method that measures software size from the perspective of functional characteristics of the software system.

The mainstream function point analysis methods include IFPUG, NESMA, GB/T 36964, COSMIC, and MARK II. The market share of IFPUG or NESMA methods exceeds 90%. The GB/T 36964 method and the NESMA detailed function point method are fundamentally equivalent to the IFPUG method.

This experiment focuses on the application of function point analysis methods to measure the software size of our project "Proactive Shield: Smart Maintenance Platform for Aero Engine Industrial Equipment" to understand the principles of the function point method and master its measurement process.

Experimental Content and Steps

Determine the Engineering Type of Software System

Our project "Proactive Shield" is classified as a **new development project**. This is an intelligent maintenance platform for aerospace engine industrial equipment that utilizes artificial intelligence and machine learning technologies to implement predictive maintenance. As a newly developed system, we need to measure all the functionalities provided to users when it is first installed.

Identify and Determine Software System Boundaries and Scope

Based on the project charter and requirements specification, the boundaries of this system include:

- User Interface: Including various monitoring dashboards, alarm notification systems, and predictive maintenance analysis tools
- Backend Services: Data processing, AI analysis engine, equipment status monitoring
- External Interfaces: Communication adaptation layer with IoT devices, interaction with country/region code data

The measurement scope includes all functionalities within the project scope, excluding hardware procurement, equipment modification, deep integration with ERP systems, and independent mobile application development.

Function Point Analysis

According to the NESMA detailed counting method, we divide the functional requirements of the project into data functional requirements and transactional functional requirements:

Data Type Function Points

Internal Logical Files (ILF):

1. User Management Data - Contains user information, role permissions, etc.
2. Equipment Management Data - Contains basic equipment information, status, location, etc.

3. Alert Record Data - Contains alert history, processing status, etc.
4. Model Management Data - Contains AI model information, parameters, etc.
5. Equipment Health Status Data - Contains real-time and historical health metrics
6. Energy Consumption Monitoring Data - Contains equipment energy consumption data
7. Predictive Analysis Data - Contains prediction results, confidence levels, etc.

External Interface Files (EIF):

1. Country/Region Code Data - Maintained by external systems
2. Sensor Device Data Stream - Provided by external IoT devices but referenced in the system

Transaction Function Points**External Inputs (EI):**

1. User Registration
2. User Login
3. User Information Management
4. Equipment Addition
5. Equipment Information Modification
6. Equipment Deletion
7. Alert Confirmation
8. Alert Deletion
9. Model Import
10. Model Selection
11. Simulation Parameter Input

External Outputs (EO):

1. Equipment Health Status Report
2. Equipment Energy Consumption Report
3. Alert Statistics Report
4. Fault Trend Report
5. Equipment Status Distribution Report
6. Life Prediction Report
7. Health Data Report

External Queries (EQ):

1. Equipment List Query
2. Equipment Details Query
3. Alert Record Query
4. Historical Data Query
5. Model List Query
6. Model Information Query

Measuring Data Function Points and Transaction Function Points

Analysis of Data Type Function Point Complexity

ILF Complexity Analysis:

ILF	DET	RET	Complexity	FP
User Management Data	14	2	Low	7
Equipment Management Data	18	3	Average	10
Alert Record Data	12	2	Low	7
Model Management Data	15	3	Average	10
Equipment Health Status Data	25	4	Average	10
Energy Consumption Data	10	2	Low	7
Predictive Analysis Data	22	3	Average	10

Table 1: ILF Complexity Analysis

EIF Complexity Analysis:

EIF	DET	RET	Complexity	FP
Country/Region Code Data	5	1	Low	5
Sensor Device Data Stream	35	4	High	10

Table 2: EIF Complexity Analysis

Analysis of Transaction Function Point Complexity

EI Complexity Analysis:

EI	DET	FTR	Complexity	FP
User Registration	8	1	Low	3
User Login	3	1	Low	3
User Information Management	12	1	Average	4
Equipment Addition	15	2	Average	4
Equipment Info Modification	18	2	High	6
Equipment Deletion	3	2	Low	3
Alert Confirmation	5	1	Low	3
Alert Deletion	3	1	Low	3
Model Import	12	2	Average	4
Model Selection	6	2	Average	4
Simulation Parameter Input	20	3	High	6

Table 3: EI Complexity Analysis

EO Complexity Analysis:

EQ Complexity Analysis:

Calculating Unadjusted Function Points

The calculation of unadjusted function points (UFP) is as follows:

- ILF Total Function Points = $7 + 10 + 7 + 10 + 10 + 7 + 10 = 61$
- EIF Total Function Points = $5 + 10 = 15$

EO	DET	FTR	Complexity	FP
Equipment Health Status Report	18	3	High	7
Equipment Energy Report	12	2	Average	5
Alert Statistics Report	10	2	Average	5
Fault Trend Report	15	3	High	7
Equipment Status Report	14	2	Average	5
Life Prediction Report	16	3	High	7
Health Data Report	14	3	Average	5

Table 4: EO Complexity Analysis

EQ	DET	FTR	Complexity	FP
Equipment List Query	12	2	Average	4
Equipment Details Query	18	3	High	6
Alert Record Query	10	2	Average	4
Historical Data Query	15	3	High	6
Model List Query	8	1	Low	3
Model Information Query	12	2	Average	4

Table 5: EQ Complexity Analysis

- EI Total Function Points = $3 + 3 + 4 + 4 + 6 + 3 + 3 + 3 + 4 + 4 + 6 = 43$
- EO Total Function Points = $7 + 5 + 5 + 7 + 5 + 7 + 5 = 41$
- EQ Total Function Points = $4 + 6 + 4 + 6 + 3 + 4 = 27$

$$\text{UFP} = \text{ILF} + \text{EIF} + \text{EI} + \text{EO} + \text{EQ} = 61 + 15 + 43 + 41 + 27 = 187$$

Calculating Adjusted Function Points

According to the IFPUG method, we use 14 General System Characteristics (GSCs) to adjust the unadjusted function points. The assessment is as follows:

General System Characteristic	Rating (0-5)
Data Communications	5
Distributed Data Processing	4
Performance	4
Heavily Used Configuration	3
Transaction Rate	3
Online Data Entry	5
End-User Efficiency	4
Online Update	4
Complex Processing	5
Reusability	3
Installation Ease	2
Operational Ease	3
Multiple Sites	2
Facilitate Change	3
Total	50

Table 6: General System Characteristics Assessment

$$\text{Calculating VAF (Value Adjustment Factor): } \text{VAF} = 0.65 + 0.01 \times 50 = 1.15$$

Calculating adjusted function points (FP): $FP = UFP \times VAF = 187 \times 1.15 = 215.05 \approx 215$

Experimental Results

Through the NESMA detailed counting method, the software size of the "Proactive Shield: Smart Maintenance Platform for Aero Engine Industrial Equipment" project was measured to be:

- Unadjusted Function Points (UFP): 187
- Adjusted Function Points (FP): 215

Verification of Results

To verify the above results, we used the GB/T 36964 method. The GB/T 36964 method is essentially equivalent to the IFPUG method but has some minor differences in detailed implementation.

The calculation results using the GB/T 36964 method:

- Unadjusted Function Points (UFP): 190
- Adjusted Function Points (FP): 219

The difference between the two methods is approximately 1.6% for UFP and 1.9% for FP, which is within an acceptable range and validates the reasonableness of our primary calculation results.

Learning Experience

Through this experiment, I gained a deeper understanding of the function point analysis method:

1. Function point analysis starts from the user's perspective, focusing on the functionality provided by the software system rather than lines of code or technical implementation details, making it an ideal tool for early size estimation.
2. Correctly identifying system boundaries is crucial, as it determines which functions should be included in the calculation and which should be excluded. In real projects, this requires thorough communication with stakeholders.
3. Distinguishing between ILF and EIF, as well as between EI, EO, and EQ, is not always straightforward and requires a deep understanding of the system.
4. Function point analysis needs to consider the number of Data Element Types (DET) and Record Element Types (RET)/File Type Referenced (FTR), which requires detailed analysis.
5. Different function point analysis methods (NESMA, IFPUG, GB/T 36964, etc.) have subtle differences, but the core principles are fundamentally the same.
6. Adjustment factors can help more accurately reflect the complexity of the system, but they also introduce subjectivity, and different analysts may arrive at slightly different results.
7. Function point analysis provides a standardized method for software size measurement, which helps with project cost estimation and comparing the size of different projects.

This experiment made me realize that software size measurement is not just a technical activity but also an analytical process that requires experience and judgment. As understanding of the project deepens, the accuracy of function point counting also improves. In future projects, I will start function point analysis earlier and continuously update and refine these measurements as the project progresses.

Problem 2

Software Cost Estimation

Experimental Purpose and Principles

Experimental Purpose

The purpose of this experiment is to understand the concept of software cost estimation and practice the process of software cost estimation. Through cost estimation of the "Proactive Shield: Smart Maintenance Platform for Aero Engine Industrial Equipment" project, this experiment aims to master the methods and techniques of software cost estimation. The experiment requires selecting a specific software cost estimation method to measure the software cost of the course project and using another software cost estimation method to verify the obtained results.

Experimental Principles

Software cost estimation is the foundation for software engineering management and software economic evaluation. The mainstream software cost estimation methods are primarily based on Function Point Analysis (FPA) methods and their results. Function Point Analysis measures software size from the perspective of functional characteristics of the software system. Currently, the mainstream function point analysis methods include IFPUG, NESMA, GB/T 36964, COSMIC, and MARK II. Among them, the market share of IFPUG or NESMA methods exceeds 90%. The GB/T 36964 method is essentially equivalent to the NESMA detailed function point method and the IFPUG method.

The basic process of software cost estimation involves first determining the software size, selecting an appropriate productivity standard, converting function points to effort, and then calculating software development costs based on personnel costs.

Experimental Results (Project Cost)

Obtaining Software Size

Based on the results of the function point analysis from the previous experiment, using the GB/T 36964 method, the software size of this project is:

- Unadjusted Function Points (UFP): 190
- Adjusted Function Points (FP): 219

Selecting an Appropriate Productivity Standard

According to the software labor productivity data published by the Beijing Software Cost Evaluation Technology Innovation Alliance (BSCEA), we selected the following productivity standard:

Application Type	Complexity	Productivity (Person-Hours/FP)
Business Application System	Medium	8.5

Table 7: Selected Productivity Standard

Our system is classified as a business application system with medium complexity, so we selected 8.5 person-hours/function point as the productivity standard.

Converting to Calendar Time According to the Productivity Standard

Calculation of project development effort:

- Effort = Number of Function Points $\times$ Productivity = $219 \times 8.5 = 1,861.5$ person-hours

Assuming 8 working hours per day and 22.5 working days per month:

- Monthly working hours = $22.5 \times 8 = 180$ person-hours
- Months required for project development = $1,861.5 \div 180 = 10.34$ months

Assuming 4 developers simultaneously participating in project development:

- Calendar time required for project development = $10.34 \div 4 = 2.59$ months, approximately 58 working days

Obtaining Software Cost

According to industry standard salary data, the average salaries for IT industry developers are:

- Junior Developer: 15,000 CNY/month
- Mid-level Developer: 25,000 CNY/month
- Senior Developer: 35,000 CNY/month

Assuming the project team composition is:

- 1 Senior Developer (Project Manager): 35,000 CNY/month
- 1 Mid-level Developer: 25,000 CNY/month
- 2 Junior Developers: $2 \times 15,000 = 30,000$ CNY/month
- Total monthly team cost: 90,000 CNY/month

Project development cost calculation:

- Labor cost = $90,000 \times 2.59 = 233,100$ CNY
- Additional costs for hardware, software, training, etc. (estimated at 15% of labor cost):
- Other costs = $233,100 \times 15\% = 34,965$ CNY
- Total project cost = Labor cost + Other costs = $233,100 + 34,965 = 268,065$ CNY

Therefore, using the GB/T 36964 method, the estimated total cost of the Proactive Shield project is approximately **268,065 CNY**.

Verifying Results Using Another Function Point Method

To verify the above results, we used the IFPUG method for estimation:

Based on the function point count obtained using the IFPUG method from the previous experiment:

- Adjusted Function Points (FP): 215

Using the same productivity standard:

- Effort = $215 \times 8.5 = 1,827.5$ person-hours
- Months = $1,827.5 \div 180 = 10.15$ months

- Calendar time for a 4-person team = $10.15 \div 4 = 2.54$ months
- Project cost = $90,000 \times 2.54 + 90,000 \times 2.54 \times 15\% = 228,600 + 34,290 = 262,890$ CNY

The difference between the two method's estimation results is: $(268,065 - 262,890) \div 268,065 \times 100\% = 1.93\%$, which is a very small difference, validating that our estimation results have high reliability.

Learning Experience

Through this experiment of using the GB/T 36964 method for software cost estimation, I gained several important insights:

1. **Applicability of National Standards:** As China's national standard, GB/T 36964 has good applicability for software projects in China. This standard has absorbed international advanced experience while combining it with the actual situation of China's software industry, making cost estimation more aligned with domestic project realities.
2. **Consistency of Function Point Analysis:** By comparing the estimation results of the GB/T 36964 and IFPUG methods, we found that the results from the two methods are very close (difference less than 2%). This indicates good consistency between different function point analysis methods, further enhancing our confidence in the estimation results.
3. **Comprehensive Consideration of Cost Factors:** Software costs include not only direct labor costs but also indirect costs such as hardware, software, and training. In actual projects, these indirect costs may account for a significant proportion of the total cost and cannot be ignored.
4. **Impact of Team Structure on Cost:** The salary differences between developers of different levels are significant, and team composition has a notable impact on project costs. By reasonably configuring the team structure (such as the ratio of senior, mid-level, and junior developers), costs can be optimized while ensuring project quality.
5. **Selection of Productivity Standards:** Choosing appropriate productivity standards has a decisive impact on cost estimation. We need to consider factors such as project type, complexity, and development team capability comprehensively, rather than simply applying industry average data.
6. **Timeliness of Estimation:** Cost estimation results will change as the project progresses. Early estimates have greater uncertainty, and the accuracy of estimates will gradually improve as requirements become clearer and design deepens. Therefore, multiple estimations should be conducted at different stages of the project.
7. **Relationship Between Size and Productivity:** Larger scale projects typically lead to decreased productivity due to factors such as increased team coordination and communication costs. This non-linear relationship should be considered in actual projects.
8. **Differences Between Domestic and International Cost Standards:** By comparing domestic and international cost data, I found that although the labor cost of software development in China is lower than in developed countries, the gap is narrowing. At the same time, the productivity of domestic software companies is continuously improving, which is changing the cost advantage.

This experiment made me deeply realize that software cost estimation is not just a technical issue but also a management issue. Accurate cost estimation is crucial for the success of a project, as it

affects investment decisions, resource allocation, schedule planning, and many other aspects of the project. By mastering scientific cost estimation methods such as GB/T 36964, a solid foundation can be laid for the success of software projects.

In future project practices, I will focus on accumulating historical data from different types of projects, refining my understanding of productivity standards, and flexibly selecting and adjusting estimation methods according to project characteristics to continuously improve the accuracy and reliability of cost estimation.

If the results obtained using two software cost estimation methods differ significantly (greater than 20%), we would adopt the following methods to address this issue:

1. **Revisit Function Point Counting:** Check the function point counting process in both methods to ensure consistent judgment of system boundaries and function complexity, eliminating the possibility of counting errors.
2. **Analyze the Impact of Adjustment Factors:** Analyze whether there are significant differences in the use of adjustment factors between the two methods, reassessing system characteristics if necessary.
3. **Verify with a Third Method:** If the difference between GB/T 36964 and IFPUG methods is too large, introduce the NESMA method as a third-party verification.
4. **Incorporate Expert Judgment:** Invite experienced project managers or cost estimation experts to participate in the assessment and modify the estimation results based on their experience.
5. **Establish a Cost Range:** Considering the uncertainty of estimation, express the result as a cost range rather than a single value, such as "project cost is between 250,000 CNY and 300,000 CNY," providing more comprehensive information for decision-making.